Boutique App

Why Checkout & Order History Were Broken, and How Each Bug Was Fixed

Prepared for: Alamgir | Project: devops-ai-playbook / boutique-microservices | Environment: Local Docker Compose (Phase A)

This document walks through everything that went wrong while wiring up the Checkout and Order History pages in your local deployment, in the order it actually happened. Six separate bugs were hiding in the code — some in the frontend, some in the backend, one in the database schema. Fixing one would reveal the next, which is why it took several rounds to get a working order flow. Every bug below follows the same shape: what you actually saw on screen, what was really happening underneath, and the exact fix that was applied.

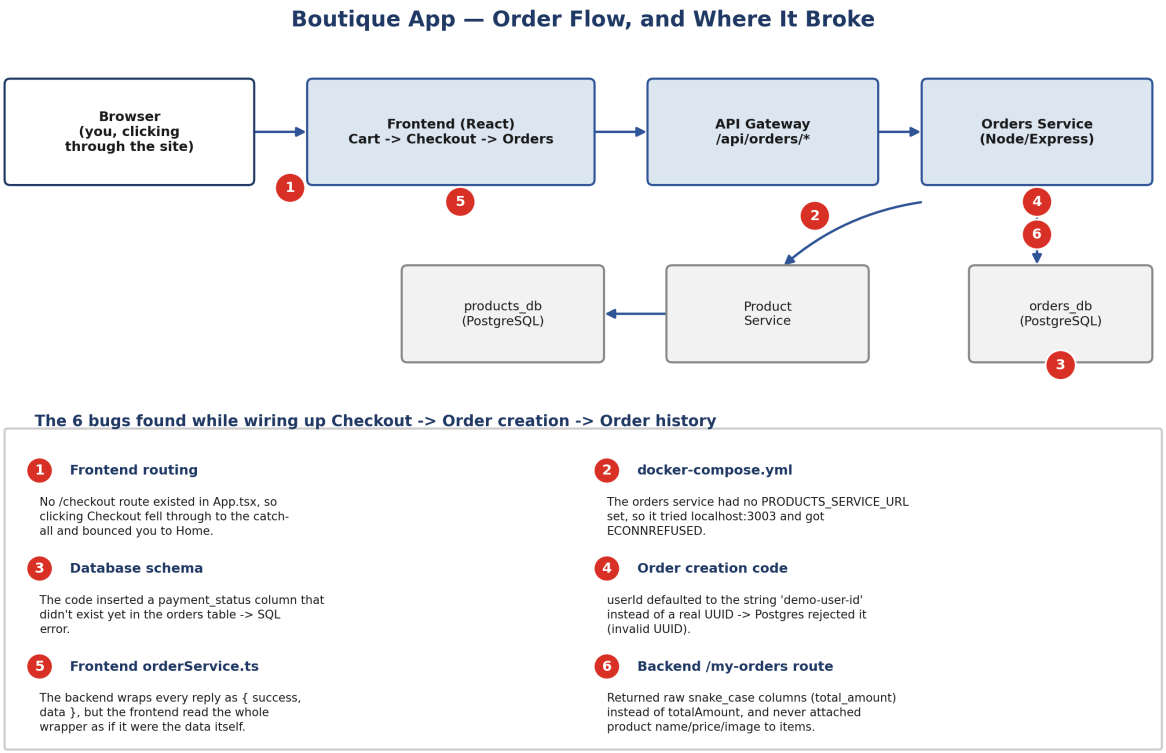

Figure 1 — The order flow across all five services, with all six bugs marked at the point where they actually lived in the system.

The Order Flow, in Plain Terms

Placing an order and viewing it later touches five different pieces, each running as its own container: the React frontend in your browser, the API Gateway that routes requests, the Orders Service that owns the orders_db database, and the Product Service that owns the products_db database. Because each service only knows about its own database, the Orders Service has to make a network call to the Product Service any time it needs a product's name, price, or image — it cannot just JOIN across databases the way a single-database app could. That one architectural fact is behind three of the six bugs below.

How the Bugs Chained Together

Every backend service in this app wraps its replies the same way: `{ "success": true, "data": ... }`. That convention is consistent and reasonable — the bugs weren't in the idea, they were in a few places where the frontend or a SQL query didn't quite line up with it. Because each broken step raised a new, different error only after the previous one was fixed, the six bugs surfaced one at a time rather than all at once.

Bug 1 — The Checkout button sent you back to the home page

What you saw: Adding items to the cart worked fine, but clicking “Checkout” instantly redirected to the home page instead of showing a checkout form.

What was actually happening:

- The React app defines its pages and URLs in `App.tsx` using a list of routes (`<Route path=... />`).
- There was simply no route registered for `/checkout` at all.
- React Router's catch-all rule ("anything unrecognized → redirect to /") caught the request and sent it home — this is normal, expected routing behavior reacting to a missing page, not a crash.

Fix:

Built a new `Checkout.tsx` page (shipping form + order summary + Place Order button) and registered it as a protected route in App.tsx.

```
<Route
  path="checkout"
  element={
    <ProtectedRoute>
      <Checkout />
    </ProtectedRoute>
  }
/>
```

Bug 2 — Placing an order failed with a connection error

What you saw: After the Checkout page existed, submitting an order returned “Failed to create order.” The orders service logs showed `ECONNREFUSED` to `localhost:3003`.

What was actually happening:

- To calculate the order total, the Orders Service has to ask the Product Service for each item's price.
- It reads the Product Service's address from an environment variable, `PRODUCTS_SERVICE_URL`.
- That variable was never set for the `orders` container in `docker-compose.yml`, so the code fell back to its default, `localhost:3003` — which inside a container means "this container itself," not the product-service container. Nothing was listening there, hence connection refused.

Fix:

Added the missing environment variable so the orders container can find the product-service container by its Docker network name.

```
orders:
  environment:
    - PORT=3005
    - PRODUCTS_SERVICE_URL=http://product-service:3003
```

Bug 3 — Placing an order failed with a database error

What you saw: Same "Failed to create order" message, but the logs now showed a different error: `column "payment_status" of relation "orders" does not exist`.

What was actually happening:

- The order-creation code inserts a row into the `orders` table that includes a `payment_status` value.
- The table itself, as originally created by the database's init script, never defined that column.
- Postgres correctly rejected the INSERT because the code and the schema had drifted apart.

Fix:

Added the missing column directly to the running database.

```
ALTER TABLE orders
  ADD COLUMN IF NOT EXISTS payment_status VARCHAR(50) DEFAULT 'pending';
```

Bug 4 — Placing an order failed with an "invalid UUID" error

What you saw: Still "Failed to create order," now with: `invalid input syntax for type uuid: "demo-user-id"`.

What was actually happening:

- The order table's `user_id` column is typed as a UUID (a specific 36-character format like `00000000-0000-0000-0000-000000000001`).
- Since the frontend wasn't sending a real logged-in user's ID yet, the backend code fell back to a placeholder string, the literal text `'demo-user-id'`.
- That text is not a valid UUID, so Postgres refused to insert it — this is the database correctly protecting its own data integrity.

Fix:

Replaced the placeholder with an actual well-formed UUID so it satisfies the column type.

```
const { items, shippingAddress,
  userId = '00000000-0000-0000-0000-000000000001'
} = req.body;
```

Bug 5 — The order actually got created, but the Orders page was a blank white screen

What you saw: No error banner, no loading spinner stuck — just an empty page where the order list should be.

What was actually happening:

- Every backend reply in this app is wrapped: `{ success: true, data: [...] }`.
- The frontend's `orderService.ts` was written as if the backend sent the array directly, so it did `return response.data;` — which actually returned the entire wrapper object, not the array inside it.
- The Orders page then tried to call `.map()` on that wrapper object to render each order. JavaScript threw a TypeError ("not a function"/wrong shape) mid-render.
- There is no error boundary on that page, so React simply rendered nothing instead of showing the crash — which is why it looked like a silent blank page rather than a visible error.

Fix:

Unwrapped one level deeper in every method of orderService.ts.

```
// before
return response.data;

// after
return response.data.data;
```

Bug 6 — The page rendered, then immediately crashed on a number

What you saw: Browser DevTools showed: `Uncaught TypeError: Cannot read properties of undefined (reading 'toFixed')`, thrown from inside the list of orders.

What was actually happening:

- This time the array itself was correct — real orders were coming through and React started rendering each one.
- But Postgres always returns column names exactly as they're defined in the table: `total_amount`, `created_at`, etc. (snake_case).
- The Orders page component was written expecting JavaScript-style camelCase: `order.totalAmount`, `order.createdAt`.
- `order.totalAmount` was therefore `undefined`, and calling `.toFixed(2)` on `undefined` throws.
- Separately, the `/my-orders` database query never fetched each item's product name, price, or image from the Product Service at all, so `item.product` was also going to be undefined the moment the page

tried to show a thumbnail.

Fix:

Rewrote the backend's /my-orders route to rename every field to the camelCase the frontend expects, and to fetch each item's product details from the Product Service before replying.

```
return {
  id: row.id,
  userId: row.user_id,
  items,                 // now includes item.product (name/price/imageUrl)
  totalAmount: row.total_amount,
  status: row.status,
  shippingAddress: row.shipping_address,
  paymentStatus: row.payment_status,
  createdAt: row.created_at,
  updatedAt: row.updated_at,
};
```

A Closer Look: the Response-Shape Bugs (5 & 6)

Bugs 5 and 6 are the two that are easiest to get confused about, because the app behaved correctly right up until the very last step of turning a database row into something the screen can draw. The diagram below lines up the broken version next to the fixed version so the difference is obvious.

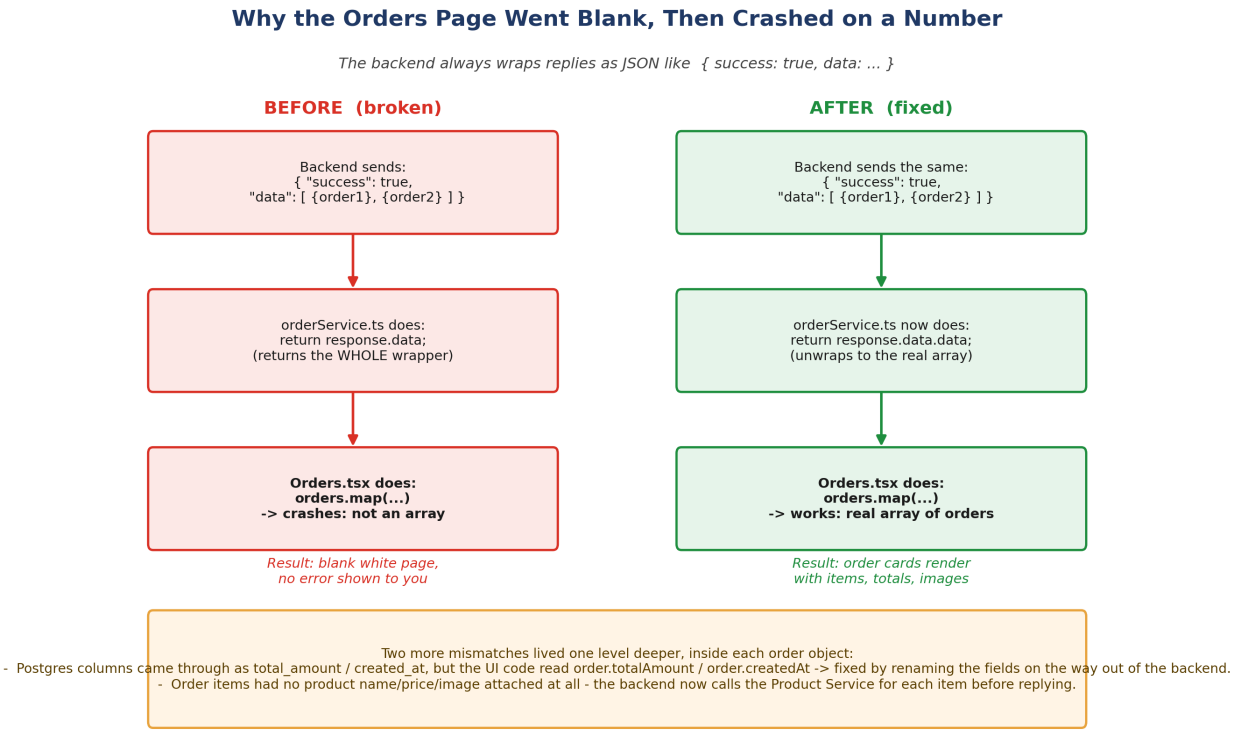

Figure 2 — Same backend response, two different frontend outcomes.

> **Why this happened:** None of these six bugs were caused by anything you did wrong while running commands. They were pre-existing gaps in the sample application's own source code — the kind of thing that only surfaces once you actually exercise a feature end-to-end, which is exactly what building and testing Checkout did.

Summary Table

#	Layer	Symptom	Root Cause	Fix
1	Frontend routing	Checkout redirected home	Missing /checkout route	Added Checkout.tsx + route
2	docker-compose.yml	ECONNREFUSED to :3003	Missing PRODUCTS_SERVICE_URL	Added env var
3	Database schema	SQL column error	payment_status column missing	ALTER TABLE
4	Order creation code	Invalid UUID error	'demo-user-id' placeholder	Real UUID default
5	Frontend orderService.ts	Orders page blank	Response wrapper not unwrapped	response.data.data
6	Backend /my-orders route	Crash on .toFixed()	snake_case fields, no product join	Rename fields + fetch product

Where Things Stand Now

All six bugs are fixed directly in your project files. The local Docker Compose environment (Phase A) now supports the full flow: browse products, add to cart, check out, and see the order in your order history with correct totals, dates, and product images. That completes Phase A end-to-end validation — the next step, whenever you're ready, is Phase B: pushing this code to your own GitHub repository and deploying it to AWS EKS.